

In one line: Every object has a hidden `[[Prototype]]` link; a read that misses its *own* properties walks the chain to `null` — objects **delegate** at run time, nothing is copied. A write lands on the object itself unless a setter or a read-only property up the chain says no. `class` wires the same chain and adds real semantics: `new`-only, fields, `#private`, `super`.

1 `class Dog extends Animal; d = new Dog()`

orange = `[[Prototype]]: Object.getPrototypeOf(x)`
blue = `.prototype`: an ordinary property of a function
d.`Speak()`: `own?` → `Dog.prototype?` → `Animal.prototype` ✓ with `this = d`

2 `new Dog(): who builds it`

plain `new F()` = the base column, with `new.target = F`

3 `o.k = v — first k found`

own data, writable	replace the value
inherited data, writable	new OWN <code>k</code> (shadow)
nowhere on the chain	new OWN <code>k</code>
data, writable:false	fail — no shadow
accessor with set	<code>set.call(o, v)</code>
accessor, no set	fail
new key, o non-extensible	fail

fail: ignored (sloppy) · `TypeError` (strict) · `false (Reflect.set) · defineProperty` skips all this

Lookup and links

- Read** `o.k`: `own?` else the `[[Prototype]]`'s, ... `null` → `undefined`; a getter found anywhere runs with `this = o` (the *receiver*).
- Write**: drawing 3. Class fields, `defineProperty` and `{ ... o }` *define*: no setter runs, an inherited read-only `k` is shadowed.
- `Object.create(p, descriptors)` links without a constructor. `Object.create(null)` or `{ __proto__: null }`: no inherited keys, no `toString` (`String(o)` → `TypeError`); `Object.groupBy` (ES2024) returns one.
- `Object.setPrototypeOf`: “very slow” in every engine (MDN); a cycle, a non-extensible object or `Object.prototype` → `TypeError`. `o.__proto__` is a *Normative Optional, Legacy* accessor on `Object.prototype`.
- `new F()` links to `F.prototype`, or to `Object.prototype` if that is not an object. Arrows, methods, `async` functions: no `.prototype`, no `new`; generators have one, still no `new`; a bound function has none — `new` uses its target's.
- `o instanceof C`: `C[Symbol.hasInstance]` if defined, else “`C.prototype` is on `o`'s chain” — fooled by `Object.create(C.prototype)` (`a #x in o` brand check is not); false for an `iframe`'s (other realm's) `array` — `Array.isArray`.

Which operation sees which keys

	inherited	non-enum.	symbol	
<code>k in o</code>	✓	✓	✓	
<code>for (k in o)</code>	✓	—	—	
<code>Object.hasOwn(o, k)</code>	—	✓	✓	ES2022
<code>Object.keys/values/entries</code>	—	—	—	as JSON.stringify
<code>{ ... o } · Object.assign</code>	—	—	✓	getters run
<code>structuredClone(o)</code>	—	—	—	deep, plain
<code>getOwnPropertyNames</code>	—	✓	—	
<code>Reflect.ownKeys</code>	—	✓	✓	

Own key order: array indices ascending, then strings, then symbols, by creation — `Object.keys({b:1, 2:1, a:1, 1:1})` is `["1","2","b","a"]`.

Descriptors and integrity

`Data {value, writable, enumerable, configurable} · accessor {get, set, enumerable, configurable}`. Literal, assignment, class field: all `true`; a *new* key via `defineProperty`: omitted → `false` / `undefined`; class methods: non-enumerable; a class's `C.prototype` is read-only. `configurable:false`: no delete, no `data↔accessor` switch; only `writable true`→`false` and (while `writable`) value may change.

	add	delete	write	sets on every own property
<code>preventExtensions</code>	—	✓	—	—
<code>seal</code>	—	—	✓	<code>configurable:false</code>
<code>freeze</code>	—	—	—	+ <code>writable:false (data)</code>

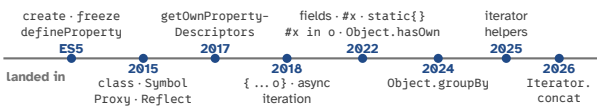
All **shallow**, setters still run; `#private` untouched. A failed assignment is silent in sloppy code, `TypeError` in strict; array methods throw in both: `Object.freeze([]).push(1)` → `TypeError`. Faithful shallow copy (`prototype`, `accessors`, `attributes`, `symbols`): `Object.create(Object.getPrototypeOf(o), Object.getOwnPropertyDescriptors(o))`.

Prototype pollution

`JSON.parse('{"__proto__":{"admin":1}}')` only makes an own key. A recursive `merge t[k] ??= {}`; `merge(t[k], s[k])` with that key writes into `t.__proto__ = Object.prototype`: every object now has `admin`. `Object.assign(t, s)` sets, so the key replaces `t`'s `prototype`; `{ ... s }` defines a plain key. Defend: null-prototype objects or `Map`, `Object.hasOwn`, `reject __proto__`, `constructor`, `prototype` keys, `Node --disable-proto=throw`.

Traps

! `F.prototype.tags = []`: one array shared by every instance — use a field.



Related: `js-scope-closures-this` · `js-types-coercion-modules` · `js-functional-idioms` · `js-engines-performance` · `ts-type-system` · `objc-interop`

Classes — what is not sugar

```
class Counter extends Base {
  static #made = 0; // ES2022: fields, #private, static
  static { /* runs once, in order; may use #private */ }
  #n = 0; // per instance; NOT a property
  step = 1; // own data property, defined
  inc = () => this.#n += this.step; // one closure per instance
  get value() { return this.#n; } // on Counter.prototype
  static is(o) { return #n in o; } // brand check
}
```

- Not sugar**: `new`-only (else `TypeError`); strict body; TDZ like `let`; methods non-enumerable, not constructible; extends `Array/Error/Map` makes real built-in instances — magic `length`, `[[MapData]]` (ES5's `Array.call(this)` could not); `map` on an `Array` subclass returns the subclass (`Symbol.species`).
- Fields**: base before its body, derived right after `super()` (drawing 2). They *define*: a base setter of the same name is not called (`this.x = 1` in a constructor would call it).
- #private**: a lexical name, not a key — invisible to `Reflect.ownKeys`, JSON, proxies, `structuredClone`; `o.#n` on an object without it → `TypeError`; `#n in o` cannot be faked, `instanceof` can; delete `this.#n` is a `SyntaxError`. A base constructor that returns `obj` lets a subclass stamp `#fields` on it, even non-extensible (Stage 3 “Non-extensible Applies to Private” makes that a `TypeError`). `static #x` read via `this` in a subclass → `TypeError`.
- super**: comes from the method's `[[HomeObject]]`, fixed where it is written: `super.m()` calls `getPrototypeOf(Home).m` on the current `this`; a copied method keeps it; `super.x = v` sets on `this`. **Mixin**: `B => class extends B { ... }`, then extends `Set(Stamped(Model))`.
- Decorators** (@dec; accessor `x = 1 = get/set` over a private slot): Stage 3 in Mar 2022, **back to Stage 2.7** on 19 May 2026 with `Decorator Metadata` — in no ES edition. `TypeScript ≥ 5.0` compiles them (no parameter decorators; `experimentalDecorators` is an older, different dialect).

Proxy + Reflect

```
const logged = t => new Proxy(t, {
  get(t, k, r) { log('get', k); return Reflect.get(t, k, r); },
  set(t, k, v, r) { return Reflect.set(t, k, v, r); }});
```

internal method	trap = Reflect....	internal	trap
<code>GetPrototypeOf</code>	<code>getPrototypeOf</code>	<code>Get</code>	<code>get</code>
<code>SetPrototypeOf</code>	<code>setPrototypeOf</code>	<code>Set</code>	<code>set</code>
<code>IsExtensible</code>	<code>isExtensible</code>	<code>HasProperty</code>	<code>has</code>
<code>PreventExtensions</code>	<code>preventExtensions</code>	<code>Delete</code>	<code>deleteProperty</code>
<code>GetOwnProperty</code>	<code>getOwnPropertyDescriptor</code>	<code>Call*</code>	<code>apply</code>
<code>DefineOwnProperty</code>	<code>defineProperty</code>	<code>Construct*</code>	<code>construct</code>
<code>OwnPropertyKeys</code>	<code>ownKeys</code>	* functions only	

`Reflect.x = trap` `x`'s default; `false` where `Object.x` throws. `r = receiver`; inherited getters see the proxy as `this`; a set trap's `false` throws in strict code. **Invariants**: no lying about a non-configurable property or a non-extensible target (→ `TypeError`). **Not forwarded**: `#private`, internal slots (new `Proxy(new Map, {})`.size → `TypeError`); `proxy != target`. `Vue 3 reactive()`: get tracks, set triggers (Vue 2's getters missed added keys).

Symbols, iterators, generators

`Symbol('d')`: a unique key; `Symbol.for('k')`: one registry for all realms. The 13 well-known symbols are protocol hooks — `iterator`, `asyncIterator`, `hasInstance`, `toPrimitive`, `toStringTag`, `species`, ...; Stage 4 for ES2027: `dispose` (using). **Iterator**: `[Symbol.iterator]() → next()` → `{value, done}`; `for..of` calls `return()` on break. A generator is a two-way iterator:

```
function* g() { const a = yield 1; const b = yield a + 1; return a + b; }
const it = g(); // runs nothing yet
it.next('x'); // {value: 1, done: false} first argument ignored
it.next(10); // a = 10 -> {value: 11, done: false}
it.next(5); // b = 5 -> {value: 15, done: true}
```

Remember

Own first, then the chain · writes shadow unless a setter or read-only says no · fields *define*, assignments *set* · `freeze` is shallow · `Reflect.x` is trap `x`'s default.